SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Y Greeshma Reddy	**Reg. No.:**	192424146
Section / Batch:	B Tech-AI&DS	**Date:**	12-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Q1. A robber plans to steal money from houses arranged in a line, but cannot rob two adjacent houses. Each house contains a certain amount of money. The goal is to maximize the total amount of money stolen without triggering the alarm. Using Dynamic Programming, determine the maximum amount of money the robber can steal.

Input Format:

- n = number of houses
- arr[] = money values in each house

Code of Conduct

I Y Greeshma Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y Greeshma Reddy

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				

Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Given an array representing money in houses, find the maximum amount that can be robbed without robbing two adjacent houses.

1. Problem Understanding

- The problem involves a robber who wants to steal the maximum amount of money from a sequence of houses.
- Each house contains a specific amount of money represented by arr[].
- The robber **cannot rob two adjacent houses**, because doing so will trigger the alarm.
- The main objective is to select the best combination of non-adjacent houses that gives the maximum total money.
- The input consists of the number of houses n and the money values stored in each house.
- A clear understanding of the restriction on adjacent houses is required before designing the solution.

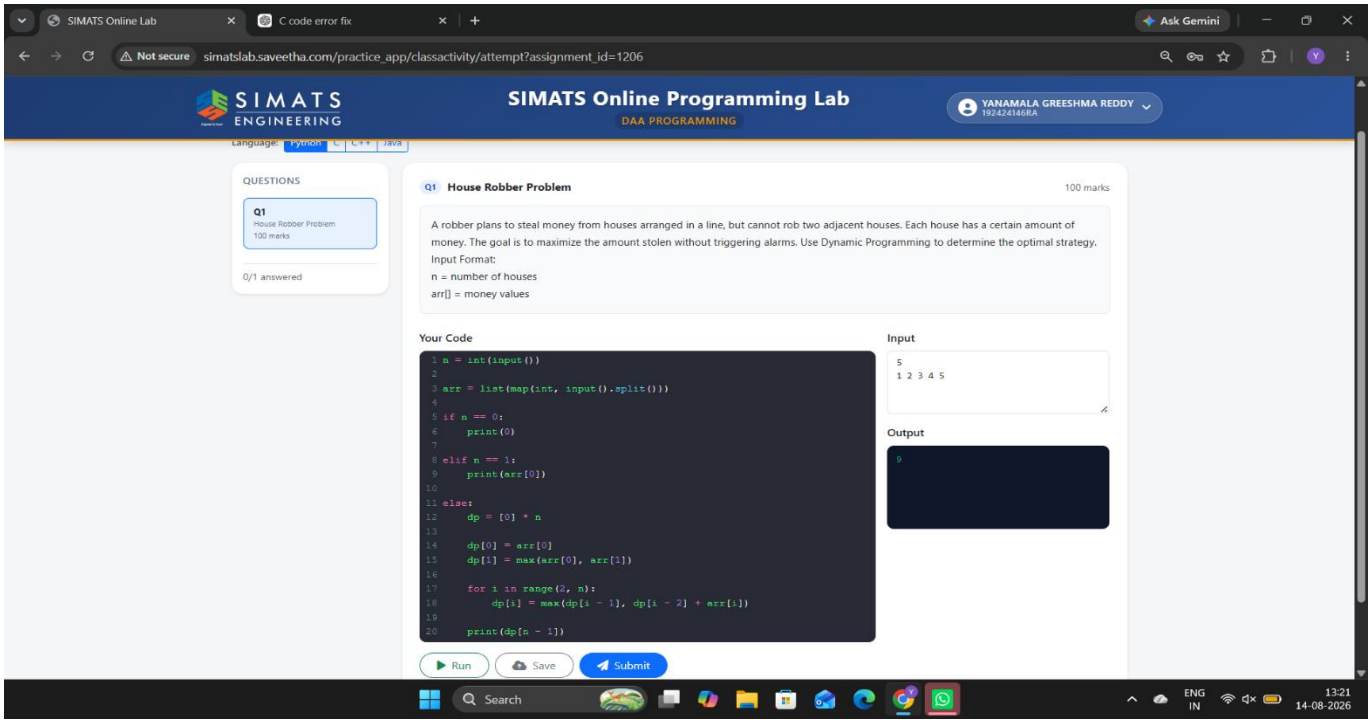
2. Algorithm

- The problem is solved using the **Dynamic Programming (DP)** technique.
- For every house, two possibilities are considered:
    - **Skip the current house** and keep the maximum amount obtained from the previous house.
    - **Rob the current house** and add its money to the maximum amount obtained up to the house before the previous one.
- The recurrence relation used is: dp[i] = max(dp[i-1], dp[i-2] + arr[i])
- The DP array stores the maximum amount that can be stolen up to each house.

- This approach avoids repeatedly solving the same subproblems and provides an efficient solution.

3. Code Implementation

- The program accepts the number of houses and the money available in each house.
- It initializes the DP array based on the first few houses.
- The program compares the amount obtained by robbing and skipping each house.
- The maximum value is stored in the DP array at every step.
- Proper conditional handling is provided for cases such as zero or one house.
- Finally, dp[n-1] gives the maximum amount of money that can be stolen.
- The implementation is simple, readable, and directly follows the Dynamic Programming algorithm.



4. Competitive Performance

- The algorithm has a **time complexity of O(n)** because every house is processed only once.
- The **space complexity is O(n)** because an additional DP array is used to store intermediate results.
- It is significantly more efficient than a brute-force approach that checks every possible combination of houses.
- The solution can efficiently handle a large number of houses.
- The algorithm provides a good balance between execution time and memory usage.
- The DP approach is suitable for competitive programming because of its linear time complexity.

5. Test Case Handling

- The program handles the case when there are **no houses (n = 0)** by returning 0.
- For **one house**, the robber simply steals the amount available in that house.
- For **two houses**, the program selects the house containing the greater amount of money.
- For multiple houses, the program correctly chooses a combination of non-adjacent houses.
- It also handles cases where skipping a house gives a better result than robbing it.
- Different money values, including equal and varying amounts, can be tested to verify the correctness of the algorithm.
- Example: For input 2 7 9 3 1, the maximum amount is **12**, obtained by selecting houses containing 2, 9, and 1.